

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 4

SUPERVISED MODELS: CLASSIFICATION CASE STUDY

Reading Time	60 minutes
Prior Knowledge	Logistic regression, Confusion matrix
Keywords	Probability of default

In the last lesson of Module 1, we are going to develop an application of a Machine Learning classification model to predict the probability of default of loan applicants.

1. Predicting the Probability of Default

The probability of default indicates the likelihood that a borrower will be unable to meet their debt payment obligations (either interest or principal payments). Default means that the lender has the legal right to attempt the recovery of their debt by seizing the borrower's assets. The higher the borrower's probability of default, the higher the interest rate charged by the lender, or the lower the amount the borrower would be eligible to obtain from the bank.

We are going to exploit real-world information on default events from a bank in Israel. In this lesson, we will develop a credit scoring model to determine the probability that a loan holder will default training a model from part of the data. We will train the model by looking at historical records of loans held and the individual characteristics of the borrowers (e.g., age, educational level, debt-to-income ratio, and other variables).

Let's read the data.

```
In [ ]: import pandas as pd

df = pd.read_csv("bank.csv")
df.head()
```

The dataset includes a sample of several tens of thousands of previously granted loans. It covers 41,188 records and 10 fields. The data includes an individual identifier and shows whether each loan defaulted or not (\$y=0\$ for no default and \$y=1\$ for default), as well as borrowers' features: age, education level (university degree, high school, illiterate,

basic education, and professional coursework), years with current employer, years in the same home, income, debt-to-income ratio, credit card debt, and other debt.

We will exploit the informativeness that several variables may have at predicting the default behavior of individuals using a logistic regression model. The model will be of use to banks or credit issuers to estimate the probability of default of an individual credit applicant with certain characteristics. A good prediction model will allow the bank to provide better and personalized financial services to customers.

The education variable is categorical with several categories. For estimation of the model, we are going to build a separate indicator variable (or "dummy variable") that shows if an individual has each education level (value of 1) or not (value of zero).

```
In [ ]: cat_vars = ["education"]
for var in cat_vars:
    cat_list = "var" + "_" + var
    cat_list = pd.get_dummies(df[var], prefix=var)
    data1 = df.join(cat_list)
    df = data1
cat_vars = ["education"]
data_vars = df.columns.values.tolist()
to_keep = [i for i in data_vars if i not in cat_vars]
```

```
In [ ]: df_final = df[to_keep]
df_final.drop(["loan_applicant_id"], axis=1, inplace=True)
df_final.head()
```

As usual, we divide the sample into a training set and a test set. The dataset includes more than 41,000 observations that we split in half to determine both samples. Default takes place in roughly 11% of the observations in the data.

```
In [ ]: X, y = (
    df_final.loc[:, df_final.columns != "y"],
    df_final.loc[:, df_final.columns == "y"],
)
print(X.shape, y.shape)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
In [ ]: print(df_final["y"].value_counts())
print(
    "Percentage of default: ",
    100
    * df_final["y"].value_counts()[1]
    / (df_final["y"].value_counts()[0] + df_final["y"].value_counts()[1]),
)
```

2. Data Preprocessing

Before training ML algorithms, we should be sure that we are proficient in the information we are working with. Obtaining some prior knowledge about how the data behaves can provide useful guidance to improve the performance of the algorithms.

2.1 Summary Statistics

Computing summary statistics (also called descriptive statistics) is a useful first task that can provide us a first glimpse at the behavior of our data. Averages and medians inform us about the representative value of each variable in our instances. Ranges and variances (or standard deviations) are informative about the dispersion of the variables and may be useful to identify outliers (more on outliers below).

Obtaining the main descriptive statistics of all the variables is very easy with pandas, as shown below. If we take a look at the means or medians of the data as measures of representativeness for each variable, we can see that the individuals in our sample are middle-aged, with long tenures in their jobs, have spent a relatively long period in the same home, and are relatively high-income individuals by Israeli standards, while they are lowly indebted (16% of income). Lastly, the sample is evenly distributed in terms of education across the five levels.

If we take a look at other features of the distribution of input features, such as dispersion, skewness, and kurtosis, we observe that the variables with the most extreme variation are those associated with the amount of debt, both credit and other types of debt. That is, there are several individuals that have extreme levels of debt in absolute terms, relative to most of the individuals.

```
In [ ]: X.describe()
```

```
In [ ]: print(X.skew())
```

```
In [ ]: print(X.kurt())
```

Let's visualize the different aspects of the distributions of each input feature to have a better understanding of the data and explore which variables are more related to default. We are going to plot the distributions of each variable on the same scale. Thus, for better visualization, we are going to standardize the data, i.e., for each input feature, we subtract its mean and divide by its standard deviation. Remember that this is one of the rescaling options we mentioned in Lesson 3.

On top of the standardization, we plot the distribution of each variable separately for those instances where we observe a default and where we do not observe it. Notice in the figure below that four variables display the most significant differences in their distributions between non-defaulting and defaulting individuals: household income, debt-to-income ratio, credit card debt, and other debt. As suggested by the descriptive statistics shown above, the level of credit card debt and other types of debt include individuals with very extreme values.

Notice also that the education attained is does not seem relevant as a default predictor since all the distributions are more or less similar across all education levels. Given this observation we may opt to exclude these variables from our model since they seem to provide little predictive ability compared to the other features. However, we should keep in mind also that there may be relevant non-linear interactions among variables that may help to predict the outcome of interest, despite seeming individually irrelevant at first. Although our Logistic Regression model will ignore the interactions between variables, we should take into account this possibility in more advanced ML models.

```
In [ ]: import matplotlib.pyplot as plt
import seaborn as sns

X_mean = X.mean()
X_std = X.std()
X_std = (X - X_mean) / X_std
X_std["y"] = y
X_std = X_std.melt(id_vars=["y"], var_name="Column", value_name="Normalized")
plt.figure(figsize=(18, 6))
ax = sns.violinplot(data=X_std, x="Column", y="Normalized", hue="y", split=True)
_ = ax.set_xticklabels(X_train.keys(), rotation=90)
```

To further explore the relationship between default and each input feature, we can perform logistic regressions where we only include one feature, on top of the constant bias term. For better visualization and comparability, now we rescale the variables using a min-max method that constrains the value of each variable into the unit interval. Notice again that four variables generate most of the action.

The analysis so far may lead us to think that we could just ignore most of the features. However, notice that the regressions above are very close to linear regressions, which neglect the presence of non-linearities in the data.

```
In [ ]: X_minmax = (X - X.min()) / (X.max() - X.min())
X_minmax["y"] = y
plt.figure(figsize=(18, 6))
g = 1
for colname in X:
    plt.subplot(3, 4, g)
    sns.regplot(
        x=colname,
        y="y",
        data=X_minmax,
        marker="",
        logistic=True,
        fit_reg=True,
        ci=None,
        label=colname,
    )
    plt.title(colname, fontsize="large")
    plt.xlabel("")
    g += 1
    # sns.lmplot(x=colname, y='y', data=X_minmax, markers = "", Logistic = True,
fig = plt.gcf()
fig.tight_layout(pad=1.0)
plt.show()
```

Another important step in data preprocessing is the analysis of correlations between features. Our prediction models may well fail if the ML algorithm is fed with many features that are strongly correlated: The optimization algorithms will face difficulties at disentangling which variable is really relevant for prediction, slowing down the training process.

Below, we display the correlation matrix of the input features, which tell us the degree of covariation between each pair of features. The variables debt-to-income, credit-card debt, and other debt have correlation coefficients above 0.5 among them. Although these correlations are not a big deal, we may have to verify the performance of ML models that exclude at least one of these variables.

```
In [ ]: X.corr()
```

In practice, some applications may involve a number of predictors that make it unfeasible to look at these simple metrics for all the variables. However, minimal verification is recommended. To further ease the analysis:

- Focus on a subset of predictors that a priori should be more relevant when predicting the outcome that we want to predict, e.g., income and debt levels seem the most reasonable in our application here.
- Track outliers in the summary statistics (more on this below).

2.2 Missing Data and Outlier Detection

There are mainly two ways to deal with missing data: removal and imputation. Removal is agnostic but costly if one whole instance is eliminated due to the presence of a single missing feature value. Imputation overcomes the loss of information but requires making assumptions, which are often erroneous.

When facing missing data, we can opt to replace it with the medians or means computed over the cross-section of assets. This implies that the missing feature value will be located within the bulk of observed values. If many values are missing, we would substantially alter the original distribution of the feature and we should assess if it makes sense to include it in our data.

In time series contexts, with views towards backtesting and prediction, the most simple imputation comes from replacing the missing information with the previous value of the time series. If we want to predict returns, however, imputation from past values should be avoided. By default, a superior choice is to set missing return indicators to zero, which is often close to the average or median. Still if a feature is highly autocorrelated, i.e., the correlation between future and past values of the variable are close to one, then imputation from the past can make sense. If not, then it should be avoided.

In time series problems, such as the prediction of future returns, we should also avoid interpolation as a way to fill in missing data. Interpolation over time periods means that we are predicting a future outcome using information that is not available at the time of the prediction. For instance, if earnings figures are disclosed in April and July,

interpolating May and June requires the knowledge of July's earnings. In May, we will not have information about July's earnings, so we cannot make predictions. In this setup, resorting to past values is a better way to go.

Sometimes, we may also detect values of input features that are extremely different from most of the data. We can opt for several heuristic methods that deal with such situations if we are concerned about the effect on the performance of our ML algorithms. These methods involve setting thresholds that determine if a value is considered an outlier:

- Any point outside the interval $[\mu - m\sigma, \mu + m\sigma]$ can be deemed an outlier. μ is the mean of the sample and σ the standard deviation. How stringent we are at labeling outliers is modulated by the multiple value m , which usually belongs to the set $\{3, 5, 10\}$.
- If the largest value is above m times the second-to-largest, then it can also be classified as an outlier (the same reasoning applied for the left side of the distribution).
- For a given small threshold q , any value outside the $[q, 1-q]$ quantile range can be considered an outlier. The range for q is usually (0.5%, 5%) with 1% and 2% being the most often used.

Once we have labeled the outliers, we must determine whether to include them or how to include them in our data. If we decide to include them, the most common practice is to replace the value of the outlier with the corresponding upper or lower threshold that we have obtained following either of the methods above.

2.3 Feature Selection and Scaling

If we have at our disposal a large set of predictors, it is reasonable to filter out redundant or unwanted input features. Simple methods include:

- Computing the correlation matrix of all input features and filter variables so that no (absolute) value is above a threshold (say, 0.7) so that redundant variables do not pollute the learning engine.
- Training linear regressions and removing the non-significant variables.
- Using unsupervised methods (clustering or principal components analysis) to retain a reduced number of features to use as inputs in the algorithm.

The methods above may overlook nonlinear relationships. Another approach would be to fit a decision tree (or a random forest) and retain only the features that have a high variable importance or exploit an autoencoder architecture. You will learn these methods in other modules and courses.

Lastly, as we stressed in the previous lesson, rescaling is needed in many ML applications. Optimization algorithms will better learn about the relevance of each input feature if they share a common scale among them. In the data analysis we performed above, we already executed to versions of feature re-scaling: normalization and min-max rescaling in the unit interval. Using `scikitlearn`, below, we again use min-max rescaling, but in the $[-1, 1]$ interval, to train our model.

```
In [ ]: from sklearn.preprocessing import MinMaxScaler

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)
```

3. Logistic Model

Now turn to train the prediction model. We do not specify any particular parameters when we define the class `logisticRegr`, meaning that we use the default optimization options. This implies that the model is regularized using ridge regularization, where the parameter α is set to 1. Please review the documentation to understand the different options at hand to refine the training of the model.

Below, we display the trained parameters of the model, which inform us about the effect of each variable on the probability of default. For instance, households with a higher debt-to-income ratio have a greater probability of default, which is intuitive from an economic standpoint. However, other parameters may make less sense at first glance. For instance, older individuals, longer tenures, or university education seem positively associated with default. Notice that this dataset provides the default behavior of individuals *conditional* on that individual having obtained a loan. For instance, older individuals usually have lower levels of debt than younger individuals. Thus, old individuals that have loans outstanding are likely to be in a worse financial condition than relatively similar young individuals. We can make similar interpretations for the remaining trained parameters.

In any case, our core task here is to develop a model that predicts well out of sample, rather than making sense of the relationship between each particular input and the probability of default. Thus, the trained parameters are of lesser importance if we can obtain a good predictive model.

```
In [ ]: from sklearn.linear_model import LogisticRegression

# parameters not specified, all are set to their defaults
logisticRegr = LogisticRegression()
logisticRegr.fit(X_train, y_train)
```

```
In [ ]: print(logisticRegr.intercept_)
for cc in range(len(X.keys())):
    print(X.keys()[cc], logisticRegr.coef_[0, cc])
```

Below, we display the accuracy of the trained model, which is roughly 92.5%. The average probability of default in the sample is 11.3%, so the model seems relatively able to discriminate some of the default cases in the test sample.

```
In [ ]: predictions = logisticRegr.predict(X_test)
```

```
In [ ]: # Use score method to get accuracy of model
score = logisticRegr.score(X_test, y_test)
print(score)
```

Because analyzing the accuracy of the model may be misleading, we are going to obtain additional performance measures that we described in the previous lesson.

First, we obtain and depict the confusion matrix. Notice now that the model fails quite terribly at predicting many of the default events. That is, the model is quite conservative and predicts in most cases that individuals will not default. This yields a low number of "false positives" but a large number of "false negatives."

```
In [ ]: from sklearn import metrics

cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9, 9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=0.5, square=True, cmap="Blues_")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
all_sample_title = "Accuracy Score: {0}".format(score)
plt.title(all_sample_title, size=15);
```

The precision of class 1 in the test set tells us out of all the default predictions made by the model, 98% were actually "bad" loan applicants. The recall of class 1 in the test set indicates the proportion of the defaulting loan applicants that our model has managed to identify as a "bad" loan applicants. So, the model only managed to identify 34% of "bad" loan applicants.

```
In [ ]: from sklearn.metrics import classification_report

print(classification_report(y_test, predictions))
```

Below, we depict the ROC curve, which tells us how the model's classification performs when we wish to increase recall, at the expense of increasing the proportion of true positives. The Area Under the Curve (AUC) is well below 1. Notice that a 0% of false positives can be obtained by only accepting around a 40% of true positives, which is close to what the trained model yields with threshold probabilities of 50%. Achieving a higher share of true positives can be obtained only at the cost of predicting more "good" individuals as bad. Thus, in an imperfect model like this, we can use our own discretion to choose if we prefer a model that over- or under-estimates the probability of default.

```
In [ ]: from sklearn.metrics import roc_auc_score, roc_curve

sns.set(style="whitegrid", color_codes=True)
logit_roc_auc = roc_auc_score(y_test, logisticRegr.predict(X_test))
fpr, tpr, thresholds = roc_curve(y_test, logisticRegr.predict_proba(X_test)[:, 1])
plt.figure()
plt.plot(fpr, tpr, label="Logistic Regression (area = %0.2f)" % logit_roc_auc)
plt.plot([0, 1], [0, 1], "r--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("Receiver operating characteristic")
plt.legend(loc="lower right")
plt.show()
```

A simple alternative to improve the prediction of default by the model is to increase its complexity using powers of the input features, as we did in Lesson 1. Adding this, we can improve the fit of the model by taking into account non-linearities in the relationship between default and the input features. For instance, the probability of default may not increase or decrease monotonically with age; it may well be lower for middle-aged individuals relative to younger or older ones.

In the block of code below, we train a logistic regression model where the input features include up to the 6th power of each input feature. In the optimization, we increase the maximum number of iterations relative to the default value of 100 due to the increased complexity of the model. Moreover, we also adjust the regularization parameter, setting "C" to 2.5. This parameter is an inverse indicator of the strength of regularization, the parameter α in Lesson 2, meaning that we are regularizing less than in the default case where it takes the value of 1. Feel free to check the documentation of the function and play around with the parameters of the model to see its performance under different settings.

The results we show below suggest a considerable improvement from adding powers of the input features. The recall increases up to 54%, which is still quite bad, but much better than the previous model. This suggests that non-linearities are relevant to improve the predictions of the model.

The relevance of these non-linearities is difficult to gauge from direct observation of the data. Other tools in machine learning, such as neural networks, are able to "identify" them in a flexible manner without the need to strictly impose functional forms and improve upon the learning ability of the models. You will study these topics in later modules.

```
In [ ]: import numpy as np

# Redefine the input feature matrix to include powers of each feature
Xpoly = X
for pp in range(2, 6):
    Xpoly = np.concatenate((Xpoly, np.power(X, pp)), axis=1)
X_train, X_test, y_train, y_test = train_test_split(
    Xpoly, y, test_size=int(len(y) * 0.5), shuffle=False
)
# Scale the features
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)
# Set up Logistic Regression
logisticRegr = LogisticRegression(C=2.5, max_iter=500)
logisticRegr.fit(X_train, y_train)
# Display coefficients
print(logisticRegr.intercept_)
print(logisticRegr.coef_)
# Compute accuracy
```

```

predictions = logisticRegr.predict(X_test)
score = logisticRegr.score(X_test, y_test)
print(score)
# Display confusion matrix and other indicators
cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9, 9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=0.5, square=True, cmap="Blues_")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
all_sample_title = "Accuracy Score: {0}".format(score)
plt.title(all_sample_title, size=15)
print(classification_report(y_test, predictions))

```

```

In [ ]: logit_roc_auc = roc_auc_score(y_test, logisticRegr.predict(X_test))
fpr, tpr, thresholds = roc_curve(y_test, logisticRegr.predict_proba(X_test)[:, 1])
plt.figure()
plt.plot(fpr, tpr, label="Logistic Regression (area = %0.2f)" % logit_roc_auc)
plt.plot([0, 1], [0, 1], "r--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver operating characteristic")
plt.legend(loc="lower right")
plt.show()

```

4. Conclusion

We have covered a complete case study to predict the probability of default using the logistic model. In the next modules, you will learn further useful machine learning techniques.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.